

ResumeIQ: An AI Powered Resume Parsing and ATS Oriented

Resume Analysis System

UBCA398J – Project

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications

by

Mohammed Azhan Palli

23BCA0288

Under the guidance of

Dr. Mohanraj G

School of Computer Science Engineering and Information Systems

VIT, Vellore



April, 2026

**ResumeIQ: An AI Powered Resume Parsing and ATS Oriented
Resume Analysis System**

UBCA398J – Project

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications

by

Mohammed Azhan Palli

23BCA0288

Under the guidance of

Dr. Mohanraj G

School of Computer Science Engineering and Information Systems

VIT, Vellore



April, 2026

DECLARATION

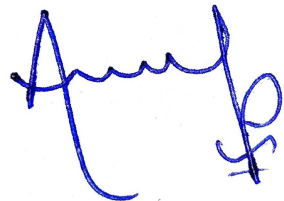
I hereby declare that the UBCA398J – Project entitled “**ResumeIQ: An AI Powered Resume Parsing and ATS Oriented Resume Analysis System**” submitted by me, for the award of the degree of *Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems* to VIT is a record of bonafide work carried out by me under the supervision of **Dr. Mohanraj G, Assistant Professor Sr. Grade 2, SCORE, VIT, Vellore.**

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 22-04-2026

Signature of the Candida

A handwritten signature in blue ink, appearing to be 'Anup', with a stylized flourish at the end.

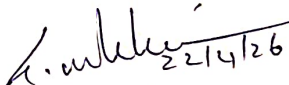
CERTIFICATE

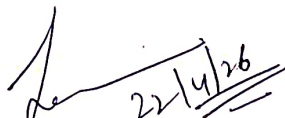
This is to certify that the UBCA398J – Project entitled “**ResumeIQ: An AI Powered Resume Parsing and ATS Oriented Resume Analysis System**” submitted by Student Mohammed Azhan Palli & 23BCA0288, SCORE, VIT, for the award of the degree of *Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems*, is a record of bonafide work carried out by him / her under my supervision during the period, 05. 12. 2025 to 17. 04. 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date: 22 / 04 / 2026


Signature of the Guide


Internal Examiner


External Examiner



Head of the Department
Department of Computer Applications

Head of the Department,
Department of Computer Applications,
School of Computer Science Engineering
and Information Systems,
Vellore Institute of Technology (VIT),
(Deemed to be university under section 3 of UGC ACT, 1956)
Vellore – 632 014.

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **UBCA398J – Project** guide **Dr. Mohanraj G, Assistant Professor Sr. Grade 2**, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for **his** constant guidance, continual encouragement, in my endeavor. My association with **him** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of **Artificial Intelligence (AI) and Natural Language Processing (NLP)**.

I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan, Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**, and Registrar **Dr. Jayabarathi T.**

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Computer Applications, **Dr. Vijayan E**, Project Coordinator **Dr. Ephzibah E.P**, SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: 22-04-2026

Mohammed Azhan Palli

Executive Summary

ResumeIQ is an intelligent tool that automatically parses, analyzes, and enhances the resume using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques.

This platform takes input in the form of resumes in PDF and DOCX formats and uses document parsing techniques to extract unstructured data from the documents in order to transform it into structured data like personal details, skills, education, projects, work experience, etc.

One of the important features of ResumeIQ is the availability of a Resume Versioning System where one can create, modify, edit, or even delete different versions of their resume. Each resume version (V) has a unique and shareable URL (URL_v) associated with it; thus, making sure that each time, recruiters get the latest version only and there is no repetition of work.

Also, the platform scores resumes based on skills relevancy, readability, and correct grammar with the help of its Applicant Tracking System (ATS) system. Further, it provides recommendations in order to enhance the overall resume.

Built using React for the frontend and FastAPI for the backend, ResumeIQ demonstrates how intelligent automation can improve recruitment efficiency, reduce manual effort, and provide accurate insights.

CONTENTS	Page No
Acknowledgement	i
Executive Summary	ii
Table of Contents	iii
List of Figures	vi
List of Tables	vii
Abbreviations	viii
Symbols and Notations	ix
1 INTRODUCTION	1
1.1 Objective	1
1.2 Motivation	2
1.3 Background	2
2 PROJECT DESCRIPTION AND GOALS	3
2.1 Problem Statement	3
2.2 Scope of the Project	4
2.3 Gap Analysis	4
2.4 Proposed System	5
3 TECHNICAL SPECIFICATION	6
3.1 Overview of Technology	6
3.2 System Architecture	7
3.2.1 User Interface Layer	9
3.2.2 API Communication Layer	9
3.2.3 Backend Processing Layer	9
3.2.4 Document Processing Layer	10

	3.2.5 AI Processing Layer	10
	3.2.6 Database Layer	10
	3.2.7 Analytics and Scoring Layer	10
	3.2.8 Authentication and Security Layer	10
	3.2.9 Caching and Optimization Layer	10
	3.2.10 Versioning and Sharing Layer	11
	3.3 Functional Requirements	11
	3.4 Non-Functional Requirements	12
	3.5 Hardware Requirements	13
	3.6 Software Requirements	13
	3.7 Data Flow Description	14
4	DESIGN APPROACH AND DETAILS	16
	4.1 Literature Survey	16
	4.2 Findings from Literature Survey	18
	4.3 UML Diagrams	19
	4.4 System Modules	20
	4.5 Design Approach / Materials & Methods	21
	4.6 Codes and Standards	22
5	SCHEDULE, TASKS AND MILESTONES	24
	5.1 Project Timeline	24
	5.2 Phase-wise Description	24

6	PROJECT DEMONSTRATION	26
	6.1 Implementation Overview	26
	6.2 System Workflow	27
	6.3 System Demonstration	28
	6.4 Sample API / Code Snippet	33
	6.4.1 API Calling	33
	6.4.2 REST Method	34
	6.5 Detailed Output Analysis	35
7	RESULT AND DISCUSSION	37
	7.1 System Performance Evaluation	37
	7.2 System Usability	38
	7.3 Cost Analysis	39
	7.4 Discussion	40
8	SUMMARY	41
	8.1 Summary of Findings	41
	8.2 Recommendations	41
	8.3 Conclusion	42
9	REFERENCES	43

List of Figures

Figure No	Title	Page No
Figure 3.1	System Architecture of ResumeIQ	8
Figure 4.1	UML Diagrams of ResumeIQ	19
Figure 6.1	Home Page	28
Figure 6.2	Resume Summary Page	28
Figure 6.3	Analysis Page	29
Figure 6.4	Analytics Page	29
Figure 6.5	Version Creation Page	30
Figure 6.6	Versions Page	31
Figure 6.7	View Resume Version Page	31
Figure 6.8	Suggestions Page	32
Figure 6.9	Resumes History Page	33

List of Tables

Table No	Title	Page No
Table 2.1	Gap Analysis of Existing and Proposed System	4
Table 3.1	Functional Requirements	11
Table 3.2	Non-Functional Requirements	12
Table 3.3	Software Requirements	13
Table 4.1	Literature Survey	17
Table 4.2	System Modules	20
Table 4.3	Versioning Features	21
Table 5.1	Project Timeline	24
Table 6.1	System Output Summary	35
Table 7.1	ResumeIQ System Performance Metrics	37
Table 7.2	Comparison with Existing Systems	38
Table 7.3	Project Cost Analysis	39

List of Abbreviations

AI	Artificial Intelligence
NLP	Natural Language Processing
API	Application Programming Interface
DB	Database
SQL	Structured Query Language
PDF	Portable Document Format
DOCX	Microsoft Word Document Format
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
ML	Machine Learning
LLM	Large Language Model
NER	Named Entity Recognition
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
ATS	Applicant Tracking System
ORM	Object Relational Mapping

Symbols and Notations

Score	Overall resume score generated by the system
S_skills resume	Skills score based on matched skills in the resume
S_read	Readability score based on text complexity
S_grammar	Grammar score based on correctness of content
M_skills	Matched skills extracted from resume
MS	Missing skills suggested by AI
R_text	Extracted resume text
DB	Structured database storing resume information
API_req	API request from frontend to backend
API_res	API response returned from backend
V	Resume Version
URL_v	Dynamic shareable URL of resume version
U	User
R	Recruiter (external viewer)

CHAPTER 1

INTRODUCTION

Artificial Intelligence (AI) is the concept of designing computer programs that emulate human capabilities in performing complex activities like natural language recognition, pattern detection, decision-making, and learning from data. The domain of Artificial Intelligence (AI) includes different fields, such as machine learning (ML), Natural Language Processing (NLP), and data analytics. Using AI technologies, computer programs are able to analyze a vast amount of unstructured data and generate useful information from it.

Over the past few years, AI has made a remarkable presence in the field of recruitment, especially when analyzing resumes and evaluating candidates. Traditional approaches in recruiting involve manually analyzing resumes, which is a lengthy process that also suffers from inconsistency and bias in decision-making. As more job applications continue to pour in, firms need more efficient methods for analyzing and selecting the best candidates. This project presents an innovative system known as ResumeIQ, which utilizes AI and ATS for parsing and evaluating resumes.

1.1 Objective

The major goal of the project is to create an AI-driven resume analysis system referred to as ResumeIQ. This tool uses NLP and data analysis technologies to automate the process of resume data extraction and processing. The ultimate goal of the proposed system is to ease up the recruitment process while saving time by converting raw resume data into useful information.

ResumeIQ analyzes resumes by extracting key information such as personal details, skills, educational qualifications, projects, and work experience. Moreover, it includes an ATS-based resume scoring system that measures resume performance in terms of such criteria as skill relevancy, readability and grammatical correctness. Finally, the system offers intelligent recommendations to optimize resume content.

1.2 Motivation

The drive behind this project can be attributed to the shortcomings that exist within the conventional resume screening process. Typically, recruitment teams conduct manual reviews of a vast amount of resumes, which is inefficient and inconsistent. This manual process is likely to lead to qualified candidates being overlooked and the possibility of biased selection in the recruitment process.

Through the advancements in technology, specifically AI and NLP, it is now possible to automate resume analysis and make the whole process more efficient. The main drive behind ResumeIQ is to create an intelligent automated solution that will ease the recruitment process.

1.3 Background

Over the last several decades, resume parsing and automated candidate evaluation technologies have evolved significantly. Initially, such tools utilized purely keyword-based approaches that were not aware of the meaning behind specific phrases and could produce incorrect outputs. Moreover, they had difficulty dealing with various types of resumes and could hardly evaluate the overall candidate profile quality.

In recent times, the development of artificial intelligence and Natural Language Processing (NLP) allowed building solutions that can perceive the context, derive insights, and structure the input information correctly. Today, ATS are extensively employed in HR. Still, many of these tools use only primitive filters. In turn, ResumeIQ leverages AI and ATS approaches, adding intelligent parsing, scoring, and recommendations into the picture.

CHAPTER 2

PROJECT DESCRIPTION & GOALS

2.1 Problem Statement

Modern recruiting processes involve employers receiving a large number of resumes for each job position offered by their organization. Such resumes are usually not structured, and the process of extracting valuable data from them requires a lot of effort. Each resume should be manually checked to make sure that the candidate is eligible for a particular job position. However, such a procedure might lead to inconsistency and discrimination, resulting in potential loss of high-quality candidates.

Conventional methods for evaluating resumes do not provide sufficient flexibility to the process since they lack standardization. Additionally, the fact that many resume evaluation techniques involve searching for keywords makes them inefficient, since they do not take into consideration context. Thus, there is a need to develop an intelligent solution that would automate such a procedure as resume data analysis and evaluation, and offer some important insights regarding the candidate's profile.

2.2 Scope of the Project

The scope of the ResumeIQ system involves the creation of a system that uses Artificial Intelligence (AI) to analyze and parse resumes. The system is designed to be able to read and extract information from resumes in popular formats like PDF and DOCX, extracting data such as personal information, skills, educational background, projects undertaken, and work experience.

In particular, the system performs resume analysis using ATS. The system can analyze resumes based on skills relevance, readability, and correctness in terms of grammar. Furthermore, the system provides suggestions about how to improve resumes through identification of missing skills and improvement in content layout.

The system is built using modern technology stacks like React and FastAPI. The system is designed as a web application to ensure ease of use and scalability. Nonetheless, the system scope is limited to analyzing resumes and does not provide other recruitment management functions like interviewing candidates.

2.3 Gap Analysis

Table 2.1 Gap Analysis of Existing and Proposed System

Aspect	Existing System	Proposed System (ResumeIQ)
Resume Processing	Manual screening	Automated parsing
Time Efficiency	Time-consuming	Fast processing
Accuracy	Inconsistent	High accuracy
Bias	Human bias present	Reduced bias
Skill Matching	Keyword-based	AI-based contextual
Feedback	Not available	AI suggestions
Data Handling	Unstructured	Structured database

2.4 Proposed System

ResumeIQ is an intelligent and automated resume analysis tool that utilizes artificial intelligence for extracting structured data from resumes.

Users can submit resumes in DOCX and PDF file formats. The system will use machine learning algorithms to extract personal details, skills, education, work experience, and projects. Extracted data will be stored in the system using SQLAlchemy.

One of the essential features of the system will be the Resume Versioning Module, which will allow users to:

- 1. Create multiple resume versions**
- 2. Edit and revise current versions**
- 3. Delete outdated versions**
- 4. Generate shareable links**

Generated links will always show the most recent version of the resume to avoid duplications and save time.

ResumeIQ will have an automated ATS score calculator based on skills matching, readability, and grammatical errors. Artificial intelligence recommendations will significantly enhance the resume's quality. Each resume version can also be exported and downloaded as a PDF for offline use and sharing.

CHAPTER 3

TECHNICAL SPECIFICATION

In this chapter, we will be talking about the technical details involved in the implementation of the proposed system, ResumeIQ. This chapter provides an overview of the technology, tools, and approaches that were employed in building this application. In addition, the chapter will discuss the architecture of the system, including the relationship between the front-end and back-end, as well as the significance of databases and APIs in making sure that there is seamless communication among all the components.

The use of AI and NLP in the process of parsing and analyzing resumes will also be discussed.

3.1 Overview of Technology

ResumeIQ is built using contemporary web and AI technologies to create a powerful, efficient and scalable solution for resume parsing and analysis. The application works under the client-server architecture when all connections between the frontend and backend are established using Representational State Transfer (REST) APIs.

The frontend is implemented using React.js technology to provide users with an engaging interface to upload their resumes and see extracted data presented in a dashboard form. The frontend has been designed to be responsive for the best user experience on all types of devices. The backend is implemented with the use of FastAPI – a library in Python that helps to implement fast and highly effective APIs. All data processing and request management will happen in the backend. The database layer has been created using SQLAlchemy, a relational database management tool that makes it possible to manage the data extracted from users' resumes efficiently. To improve the intelligence level of the software, AI and NLP methods are utilized. These include the utilization of libraries like spaCy for processing text content and skills, as well as AI algorithms for analyzing resumes and providing suggestions. Besides, the application also uses scoring functions to score the quality of resumes by considering factors like readability and grammar.

3.2 System Architecture

ResumeIQ uses a modular and scalable design that incorporates front-end, back-end, and artificial intelligence-driven processing modules. This system is capable of handling uploads of resumes, processing of documents, structured data extraction, and insights generation through an organized process.

The design involves several layers such as the user interface layer, back-end API layer, document processing layer, AI processing engine layer, database management system layer, and analytics layer. Each layer plays a particular role and interacts with other layers through specified interfaces.

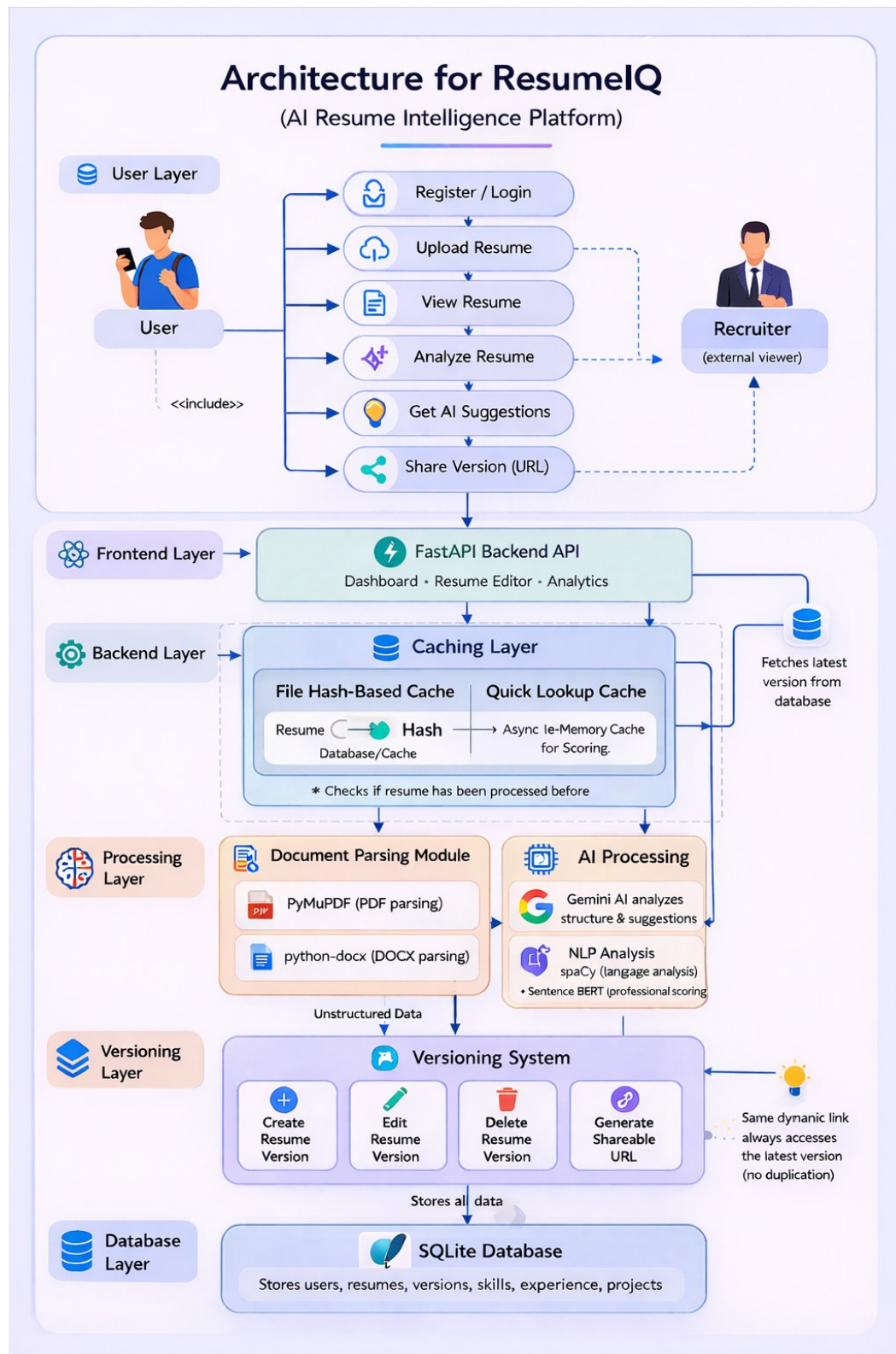


Fig. 3.1 System Architecture of ResumelQ

Multiple components interconnect and form the system architecture of ResumeIQ. The process starts from the user uploading their resume via the frontend web interface, which is developed using the React framework. The resume will be uploaded to the backend by means of Hypertext Transfer Protocol (HTTP) requests using the FastAPI server. The backend works as the central component in charge of handling data transfer and integration of various modules. The uploaded resume moves on to the document parser. As per the document type, the system relies either on PyMuPDF (for PDF files) or python-docx (for DOCX files) to extract the raw text data contained in the document, which is used further for analysis.

Raw text is analyzed within the AI module, which makes use of language models like Gemini AI to extract structured data such as personal data, skills, education, projects, and work experience. Besides, NLP techniques are used with help of spaCy library. The ResumeIQ system follows a layered architecture design, where each layer is responsible for a specific set of functionalities. This separation of concerns improves system scalability, maintainability, and performance.

3.2.1 User Interface Layer:

This layer represents the frontend of the system developed using React. It provides an interactive interface for users to upload resumes, view extracted information, analyze scores, and receive suggestions. All user interactions are initiated from this layer.

3.2.2 API Communication Layer:

This layer acts as a bridge between the frontend and backend. It handles HTTP requests and responses, ensuring secure and structured communication between the user interface and backend services.

3.2.3 Backend Processing Layer:

The backend is implemented using FastAPI, which manages business logic, request handling, and coordination between different modules. It processes resume data and controls the flow of information within the system.

3.2.4 Document Processing Layer:

This layer is responsible for extracting text from uploaded resumes. Libraries such as PyMuPDF and python-docx are used to process PDF and DOCX files and convert them into raw text format.

3.2.5 AI Processing Layer:

The AI layer uses NLP techniques to analyze the extracted text. It identifies key information such as skills, education, and work experience. It also generates insights and suggestions for improving the resume.

3.2.6 Database Layer:

This layer handles data storage using SQLAlchemy and SQLite. It stores structured resume information, enabling efficient retrieval and management of data.

3.2.7 Analytics and Scoring Layer:

This layer evaluates resumes using an ATS-based scoring mechanism. It calculates scores based on skills, readability, and grammar, and generates feedback for users.

The layered architecture ensures that each component operates independently while maintaining smooth interaction with other components, resulting in an efficient and scalable system design. The architecture ensures scalability, modularity, and separation of concerns.

3.2.8 Authentication and Security Layer:

This layer handles user authentication and authorization using JWT (JSON Web Tokens). It ensures that only authorized users can access and manage their resumes. Passwords are securely stored using hashing techniques.

3.2.9 Caching and Optimization Layer:

This layer improves system performance by implementing file hash-based caching and in-memory caching. It prevents duplicate processing of resumes and reduces response time significantly. This layer is placed between the backend processing layer and AI/document processing modules to intercept duplicate requests and improve performance.

3.2.10 Versioning and Sharing Layer:

This layer enables users to create multiple versions of their resumes. Each version is assigned a unique shareable URL, allowing users to share customized resumes publicly without requiring authentication.

3.3 Functional Requirements

Table 3.1 Functional Requirements

Requirement ID	Description
FR1	The system shall allow users to upload resumes in PDF and DOCX formats
FR2	The system shall extract personal information from resumes
FR3	The system shall identify and extract skills from resumes
FR4	The system shall extract education and work experience details
FR5	The system shall store extracted data in a structured database
FR6	The system shall evaluate resumes using ATS-based scoring
FR7	The system shall provide AI-based suggestions for improvement
FR8	The system shall display analysis results through a user dashboard
FR9	The system shall validate uploaded files before processing
FR10	The system shall allow users to register and login securely
FR11	The system shall associate resumes with specific users
FR12	The system shall support resume versioning and sharing via unique links
FR13	The system shall implement caching mechanisms (file hashing and in-memory cache) to avoid redundant processing and improve performance
FR14	The system shall allow users to download resume versions as PDF files

Functional Requirements These are the basic functionalities that the ResumeIQ system should have. They explain the actions that the system should undertake to meet its desired goals. ResumeIQ provides users with the opportunity to upload their resumes in compatible formats such as PDF and DOCX. It then parses the uploaded resume to derive important details regarding the user's personal information, skill sets, educational background, and professional experience. All the collected data is saved in an organized database.

The system has an ATS-based scoring feature, which grades the resume according to a number of parameters. In addition, it provides AI-based recommendations to enable users to enhance their resumes.

3.4 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

Requirement Type	Description
Performance	The system should process resumes efficiently with minimal response time
Scalability	The system should handle multiple users and large volumes of resumes
Usability	The interface should be user-friendly and easy to navigate
Reliability	The system should provide consistent and accurate results
Security	User data should be securely stored and protected
Maintainability	The system should be easy to update and extend
Availability	The system should be accessible whenever required

The non-functional requirements define the quality attributes of the ResumeIQ system. These requirements ensure that the system performs efficiently, securely, and reliably under different conditions.

The system is designed to provide high performance by processing resumes quickly and delivering results with minimal delay. It is scalable to handle multiple users and large datasets without performance degradation. The user interface is developed with a focus on usability, ensuring that users can easily interact with the system.

Security is an important aspect, and the system ensures safe handling of user data. Reliability is maintained by providing consistent and accurate analysis results. Additionally, the system is designed to be maintainable and flexible, allowing future enhancements and updates. The system also ensures high availability so that users can access it whenever needed.

3.5 Hardware Requirements

There will be no requirement of any specific hardware configuration in the ResumeIQ system, and it can be created and run on regular computing machines. The hardware specifications needed for the system are:

It needs a computer having at least 4 GB of RAM, but more than 8 GB is preferred while running the system. A multi-core CPU such as Intel i5 or similar is good enough to process back-end tasks along with artificial intelligence operations.

At least 10 GB of free space is needed to store application files and database entries along with uploading the resume. An active internet connection is also required for effective communication between frontend and backend along with using AI services. Hence, the hardware requirements remain modest for the system.

3.6 Software Requirements

Table 3.3 Software Requirements

Software	Description
Operating System	Windows / Linux / macOS
Frontend	React.js
Backend	FastAPI (Python)
Database	SQLite
ORM	SQLAlchemy
NLP Library	spaCy
Document Processing	PyMuPDF, python-docx
AI Model	Gemini AI
Programming Language	Python, JavaScript

The technology stack used for building the ResumeIQ system is contemporary and allows efficient implementation and deployment of the system. React.js was employed for the creation of the front-end part of the system and ensures a dynamic user interface. On the other hand, the back-end part of the system is implemented using FastAPI – a fast Python framework that facilitates effective interaction between APIs.

The system employs SQLite as the database, allowing structured storage of resume data. The SQLAlchemy library acts as the object-relational mapper. In turn, NLP is performed via the spaCy library. The parsing process is conducted via PyMuPDF for PDF files and python-docx for DOCX documents.

Finally, the system possesses AI features by employing Gemini AI for generating insights and recommendations. All components of the system are created via programming languages like Python and JavaScript.

3.7 Data Flow Description

Data is transmitted through several steps of operations within the ResumeIQ software efficiently. The process starts when the user signs into the application and provides a resume that goes through the frontend interface. In this case, the resume file is transmitted to the server via HTTP requests.

Afterward, the backend first checks the caching layer using file hashing to determine whether the resume has already been processed. If a match is found, the system directly retrieves structured data and analysis results from the cache without invoking document parsing or AI processing modules. If not, the application receives the resume file and extracts raw text from it with the help of specific libraries (such as PyMuPDF for PDF files or python-docx for DOCX files). The received data is further sent to the AI/NLP module for processing.

Within this module, the text is processed, and certain structured data is identified (personal information, skill list, education, projects, and work experience). All the extracted structured data is checked and saved to the database with SQLAlchemy.

The data is processed with ATS scoring, where the resume's readability and grammatical aspects are evaluated based on the skill set relevance. Moreover, there is also the version management capability within the system whereby the user can handle different versions of his resume. Every version of the resume is stored on the database but is connected using a unique dynamic URL. Once accessed by a third party outside the system, the current version of the resume will be retrieved from the database. Each resume version (V) is associated with a dynamic URL (URL_v), which is used by the recruiter (R) to access the latest resume data stored in the database (DB).

Finally, the analyzed data, scores, and suggestions are sent back to the frontend and displayed on the user dashboard. Users can also manage their resumes and create multiple versions, which can be shared using unique public links.

CHAPTER 4

DESIGN APPROACH AND DETAILS

In this chapter, the design process of the ResumeIQ system will be discussed along with its implementation strategy. This chapter discusses the methodologies adopted for the design process of the ResumeIQ. This chapter starts off with a review of relevant literature on resume analysis systems already available. It also describes the design of the system along with data flow among various modules in the system. The main purpose of this chapter is to give a brief idea about the design process of the system.

4.1 Literature Survey

This section summarizes the existing studies and systems on resume parsing and analysis. It describes their methods, strengths, and weaknesses. This literature review provides insights into recent developments and identifies areas where the new ResumeIQ system can be applied.

Table 4.1 Literature Survey

SNO	TITLE	MERITS	DEMERITS
1	Resume Parser and Auto-Formatter Using NLP (2021)	Enables automatic extraction of structured resume information using NLP techniques. Improves consistency and formatting.	Performance may vary for highly unstructured or creative resume layouts.
2	Resume Parser Using NLP Techniques for Recruitment (2019)	Efficiently extracts key resume sections such as skills, education, and experience. Reduces manual screening effort.	Semantic understanding can be further enhanced for better contextual analysis.
3	Automatic Resume Parsing Using Named Entity Recognition	Accurately identifies entities like job roles, organizations, and qualifications. Improves resume information structuring.	Requires proper training data for optimal extraction performance.
4	AI-Based Resume Screening System	Automates candidate shortlisting and ranking using intelligent models. Improves recruitment efficiency.	Model performance depends on dataset quality and feature selection.
5	Intelligent Recruitment System Using Artificial Intelligence	Supports faster hiring decisions and reduces recruiter workload.	System explainability can be improved for better decision transparency.
6	Automatic CV Ranking Using Machine Learning	Ranks resumes based on similarity with job requirements. Enhances relevance in candidate selection.	Requires continuous tuning for different job domains.
7	Multi-Lingual Resume Parsing Using Deep Learning	Enables parsing of resumes in multiple languages. Useful for global recruitment scenarios.	Model training may require higher computational resources.
8	ATS-Compliant Resume Generation Using Keyword Optimization	Improves resume visibility in ATS through keyword analysis.	Excessive keyword optimization may affect natural readability.
9	Skill Matching Framework Using NLP	Matches candidate skills with job descriptions effectively. Improves resume filtering accuracy.	Depth of experience analysis can be further incorporated.
10	Deep Learning Based Resume Classification System	Automatically categorizes resumes into job roles. Improves recruitment automation.	Requires large datasets for better classification accuracy.
11	Topic Modeling Based Resume Understanding	Identifies hidden skill patterns and domain expertise. Helps recruiters understand candidate strengths.	Topic interpretation may need manual validation.
12	Semantic Similarity Based Resume Evaluation	Measures similarity between resume content and job descriptions. Enhances ATS filtering performance.	Sensitive to incomplete or noisy resume text.
13	NLP Driven Resume Recommendation System	Provides suggestions to improve resume quality. Supports candidate profile enhancement.	Suggestion relevance depends on extraction accuracy.
14	Machine Learning Techniques in Recruitment Systems (2017)	Enables predictive analytics for recruitment decision making. Improves efficiency.	Model generalization across industries can be further explored.
15	Deep Neural Network Based Resume Parsing	Handles complex resume structures and improves extraction accuracy.	Implementation complexity and training time may increase.

4.2 Findings from Literature Survey

From the review of the literature, it can be seen that there is a growing significance of artificial intelligence and NLP in recruiting people. The current solutions mostly concentrate on the extraction of structured data, such as skills, education, and experience from resumes through NLP and machine learning algorithms.

Many research papers prove that the process of resume screening with automated technologies saves much time and effort. Such approaches as Named Entity Recognition (NER), semantic similarity analysis, and machine learning classification methods have been successfully used for resume parsing.

Nevertheless, there are some drawbacks in the current solutions. Firstly, keyword-based resume extraction does not include any contextual analysis, thus, providing unreliable results. Secondly, dealing with different formats of resume and unstructured data is still a difficult task. Finally, some solutions involve large datasets and intensive computer power.

Additionally, all current approaches consider either resume parsing or ranking alone but do not provide any kind of detailed feedback or advice regarding the resume's quality. There are no approaches that unite all of these actions into one system. The offered ResumeIQ approach tries to overcome these drawbacks by combining artificial intelligence-based resume parsing with scoring and intelligent advice, provided by an ATS. The use of NLP tools together with new web technologies allows the development of a more efficient solution.

4.3 UML Diagrams

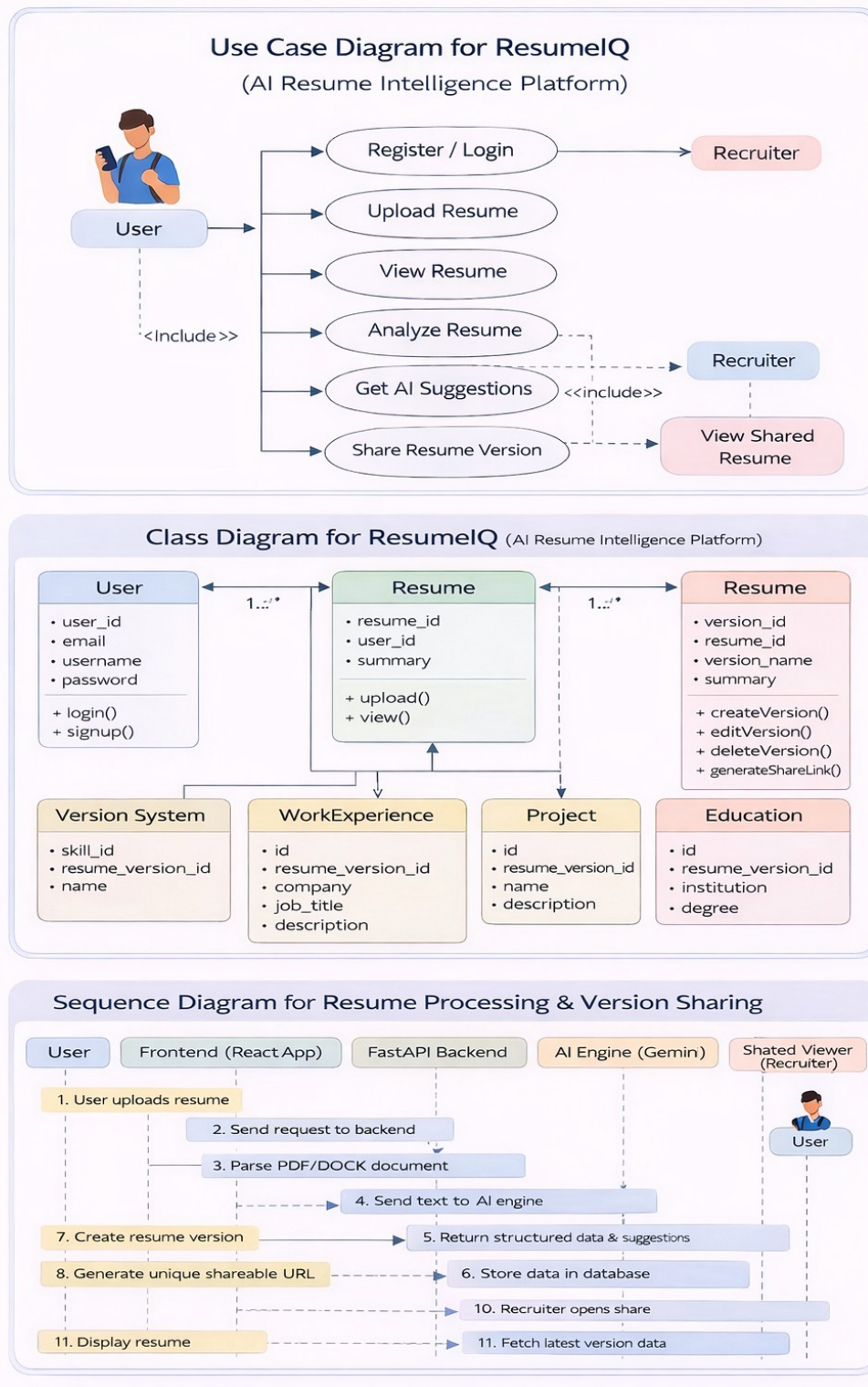


Fig. 4.1 UML Diagrams of ResumelQ

The UML diagram shown in Fig. 4.1 represents the overall structure and interaction between different components of the ResumeIQ system. The system mainly consists of multiple modules such as the User, Resume, Version System, Work Experience, Project, and Education modules. The user interacts with the system by uploading resumes and managing different versions of their resume. Each resume is processed and associated with various entities such as skills, education details, projects, and work experience. The Version System module allows users to create, edit, and manage multiple versions of their resumes, each linked with a unique identifier. The relationships between these modules illustrate how data flows within the system. For example, a resume version is linked to multiple components such as skills, projects, and education records, ensuring structured storage of information. Overall, the UML diagram provides a clear understanding of the system design, showing how different components interact to support resume parsing, analysis, and version management.

4.4 System Modules

Table 4.2 System Modules

Module	Features	Description
Resume Upload Module	File Upload (PDF/DOCX)	Allows users to upload resumes for processing
Resume Parsing Module	Text Extraction, NLP	Extracts structured data from resumes
Skill Extraction Module	Skill Identification	Identifies relevant skills using NLP techniques
Scoring Module	ATS Score Calculation	Evaluates resumes based on skills, readability, and grammar
Suggestion Module	AI Recommendations	Provides suggestions to improve resume quality
Authentication Module	User Login/Signup	Handles secure user authentication using JWT
Caching Module	File Hashing, Memory Cache	Avoids duplicate resume processing by using file hashing and improves response time using in-memory caching
Versioning Module	Resume Versions	Allows users to create and share multiple resume versions
Database Module	Data Storage	Stores extracted resume data using SQLAlchemy
API Module	RESTful APIs	Handles communication between frontend and backend
User Interface Module	Dashboard, Visualization	Displays results and insights to users

Table 4.3 Versioning Features

Feature	Description
Create Version	Allows user to create multiple resume versions
Edit Version	Modify existing resume content
Delete Version	Remove outdated versions
Share Version	Generate unique public URL
Dynamic Update	Always shows latest version data

4.5 Design Approach / Materials & Methods

ResumeIQ's modular and scalable design ensures efficient coding, easy maintenance, and future extensibility. The application uses a client-server model wherein the frontend and backend use RESTful API calls.

The frontend uses React to develop the application interface, which enables users to upload their resumes and view extracted data and analysis. The backend utilizes FastAPI to handle requests and process them according to the application's business logic while enabling communication among various modules.

ResumeIQ employs a pipeline-based approach for processing uploads. The first step involves uploading documents via the interface. Then, the backend verifies the uploaded document's file type and checks for duplicates by performing file hashing. In case of duplicates, results from previous analyses are retrieved from the cache to speed up response time. This optimization significantly reduces computational overhead and ensures efficient utilization of AI processing resources.

In cases where there are no duplicates, parsing of the document is done using PyMuPDF in the case of PDF files and python-docx in the case of DOCX files. The parsed content is then subjected to AI and NLP methods and tools such as spaCy and other AI models to obtain data in structured format.

Finally, the extracted data undergoes validation using Pydantic schemas and storage in structured format using SQLAlchemy. ResumeIQ also incorporates authentication using JSON Web Tokens (JWT). The resume evaluation module calculates ATS-based scores based on skill relevance, readability, and grammar. AI-based suggestions are generated to help users improve their resumes.

Additionally, the system supports resume versioning, allowing users to create multiple customized versions of their resumes and share them through unique public links. This design ensures modularity, scalability, and efficient system performance.

4.6 Codes and Standards

The development of the ResumeIQ system is done in line with best practices for coding that include following coding guidelines and industry standards to ensure code quality and scalability. For the backend, Python language is used to develop it following the PEP 8 coding style, which offers guidelines on how to write proper codes.

RESTful API design principles have been considered in developing APIs for communication purposes. Different endpoints have been established to carry out particular functions such as uploading of resumes, extracting data, and retrieving results. Appropriate usage of methods like GET and POST has been done to facilitate smooth communications

Relational databases design has been implemented using SQLAlchemy in designing the database. Normalization process has been used effectively to eliminate any form of data redundancy and achieve data consistency. Relations between different entities including resume, skills, education, and projects have been clearly outlined. The frontend development using React follows component-based architecture, ensuring reusability and modularity. Code is structured into reusable components, making the interface easy to maintain and extend.

In addition, error handling and validation are implemented using Pydantic schemas, ensuring that the data processed by the system is accurate and consistent. Overall, adherence to coding standards and best practices ensures that the ResumeIQ system is robust, maintainable, and scalable.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Timeline

Table 5.1 Project Timeline

Phase	Activity	Duration	Deliverable	Status
1	Research & Requirement Analysis	Dec 2025	Problem Statement, SRS	Completed
2	System Design	Jan 2026	Architecture Diagram, DB Design	Completed
3	Backend Development	Feb 2026	FastAPI APIs, Database Setup	Completed
4	Frontend Development	Feb 2026	React UI Implementation	Completed
5	AI Integration & Scoring	Mar 2026	Resume Scoring Module	Completed
6	Testing & Optimization	Mar 2026	Testing Reports	Completed
7	Documentation & Submission	Apr 2026	Final Report	Completed

5.2 Phase-wise Description

Phase 1 – Research and Requirement Analysis (December 2025):

A detailed research was performed in this phase. Problem statement was formulated and functional and non-functional requirements were defined. Additionally, a gap analysis was done to identify weaknesses in current solutions.

Phase 2 – System Design (January 2026):

In this phase, design was performed for frontend, backend, and databases. UML and data model diagrams were drawn to depict behavior and structure of the system.

Phase 3 – Backend Development (February 2026):

Backend was coded with FastAPI framework and database models were coded with SQLAlchemy. Resume upload, parsing and data access APIs were developed.

Phase 4 – Frontend Development (February 2026):

Frontend of the system was coded with React.js framework. Various features like resume upload, data display, dashboard visualization etc. were coded.

Phase 5 – AI Integration and Scoring (March 2026):

In this phase, NLP techniques and machine learning algorithms were used to identify skills and provide scores. A scoring system was made to generate resume scores according to ATS standards.

Phase 6 – Testing and Optimization (March 2026):

Testing was performed to ensure that system works perfectly and does not have any flaws. Performance improvements were also made at this stage.

CHAPTER 6

PROJECT DEMONSTRATION

6.1 Implementation Overview

ResumeIQ is realized as a full-stack application consisting of frontend, backend, and AI-driven processing components. The system allows users to upload their resumes, extract necessary information, analyze the quality of their resumes, and get recommendations on how to improve them.

The frontend part of the system is designed using React, offering a user-friendly platform for uploading resumes, analyzing resume data, and navigating various functionalities. The backend is implemented using FastAPI, processing API requests and coordinating the interaction of various parts of the system.

In the current version of the system, PDF and DOCX files can be processed by ResumeIQ using document parsers like PyMuPDF and python-docx. The content of resumes is analyzed using AI and NLP techniques to detect personal data, skills, education, work experience, etc. In order to implement user authentication in the system, JWT is used. This technology guarantees safe access to the website and secure personal data handling. All resumes are associated with the particular user account.

For better performance, file hashing and caching methods are applied in the system. If a file is processed previously, the result will be obtained from cache almost instantly. The system also supports resume versioning, enabling users to create multiple customized versions of their resumes and share them through unique public links. The evaluation module calculates ATS-based scores and generates AI-based suggestions, which are displayed on the user dashboard. The caching mechanism is implemented at the backend level before triggering AI and NLP processing.

6.2 System Workflow

ResumeIQ operates under an efficient workflow model. First, users will sign in to the website and upload their resume via the web portal. The system will then validate the file format before proceeding with its processing.

The software will check for any previous processing of the same resume using file hashing. In case of a match, the system will use the cached output of the resume to save time and increase efficiency.

For newly submitted resumes, the backend will extract raw text data using specialized libraries such as PyMuPDF or python-docx. Then, using AI and NLP, ResumeIQ will convert the text to structured data containing details such as personal information, skills, education history, and work experience.

After the data extraction, the results will be saved to the database and associated with the user. The resume will undergo evaluation using an ATS-based scoring system, determining the scores of various aspects like relevancy of skills, readability, and grammar. Additionally, users can create different versions of their resumes and distribute them using unique public URLs. Ultimately, all outcomes including data extraction, scores, and recommendations will be presented to the user dashboard.

6.3 System Demonstration

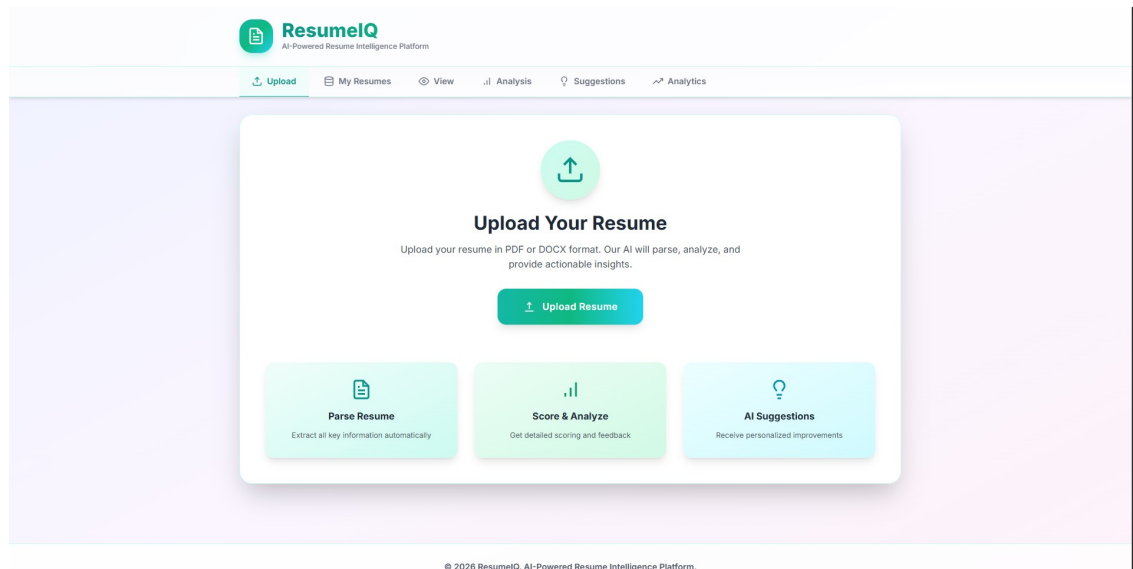


Fig. 6.1 Home Page

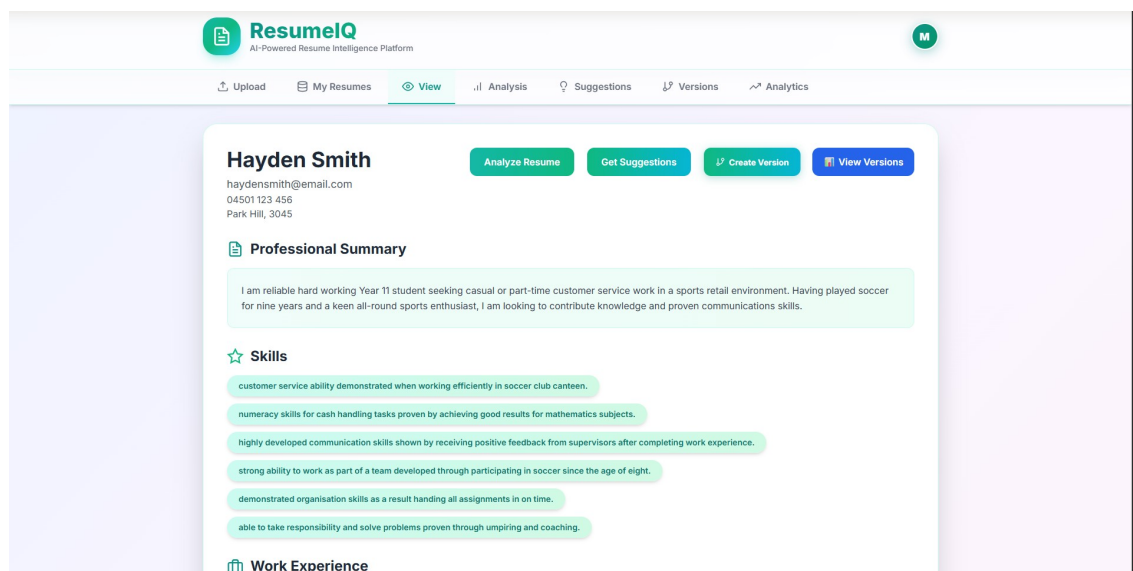


Fig. 6.2 Resume Summary Page

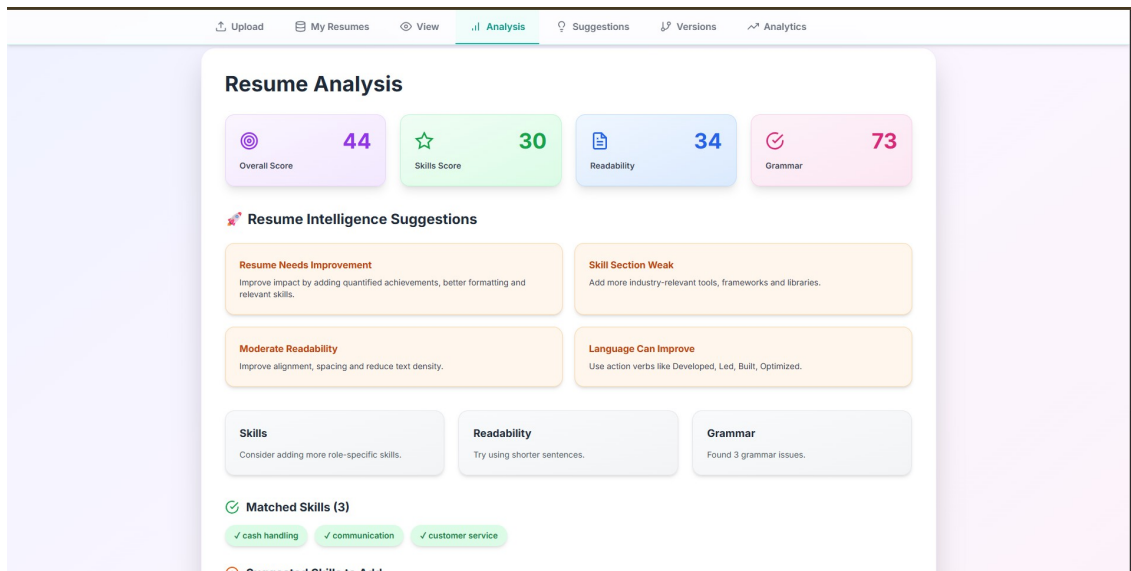


Fig. 6.3 Analysis Page

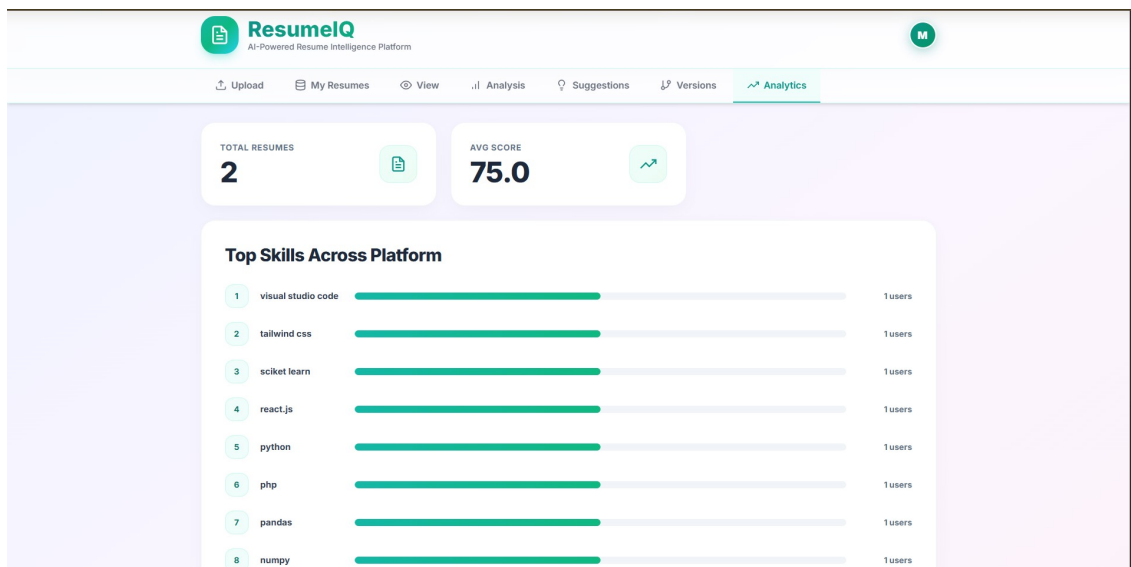


Fig. 6.4 Analytics Page

Create Resume Version

Tailor and share a snapshot for a specific role

Basic Details

Version Name *

My Version

Professional Summary

I am reliable hard working Year 11 student seeking casual or part-time customer service work in a sports retail environment. Having played soccer for nine years and a keen all-round sports enthusiast, I am looking to contribute knowledge and proven communications skills.

Skills (comma-separated)

customer service ability demonstrated when working efficiently in soccer club canteen., numerai

Work Experience

+ Add New

Entry #1

X Remove

Company

Park Hill Soccer Club

Job Title

Customer service (volunteer)

Start Date

2016-12

End Date

2017-03

Description

Fig. 6.5 Version Creation Page

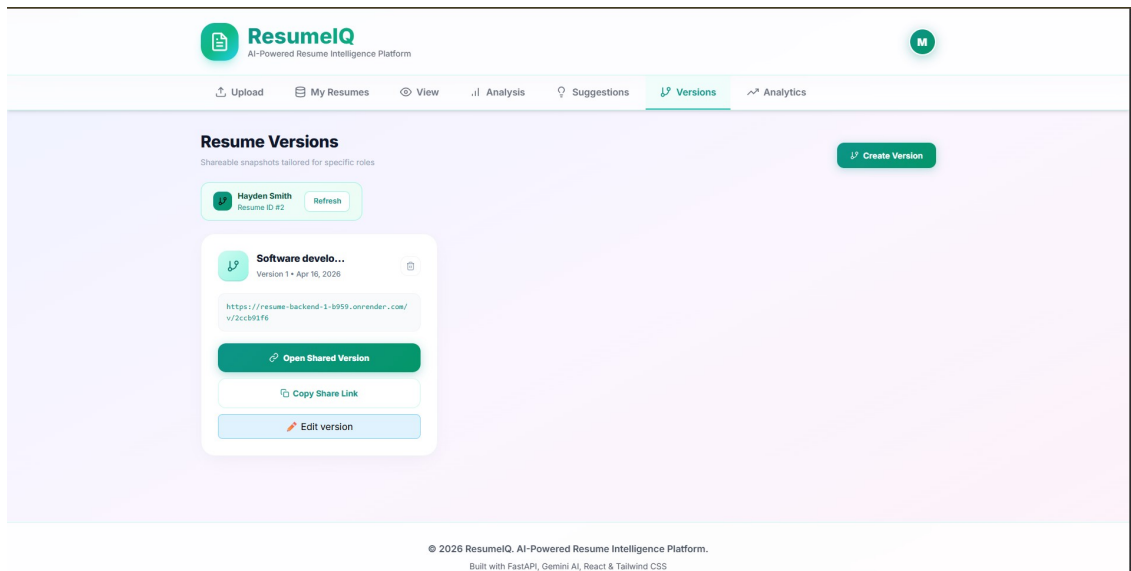


Fig. 6.6 Versions Page

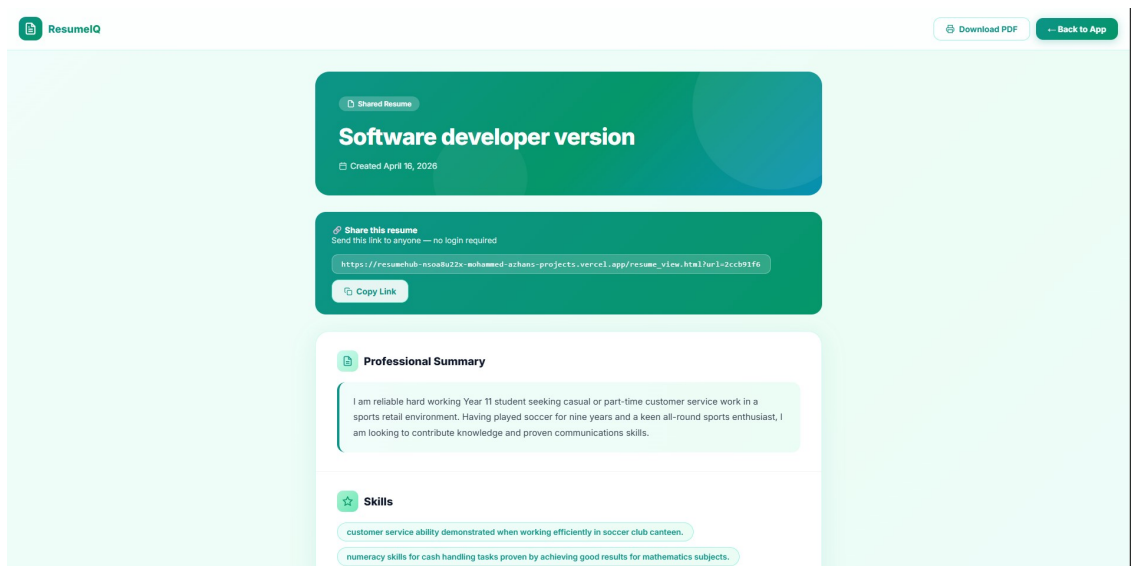


Fig. 6.7 View Resume Version Page

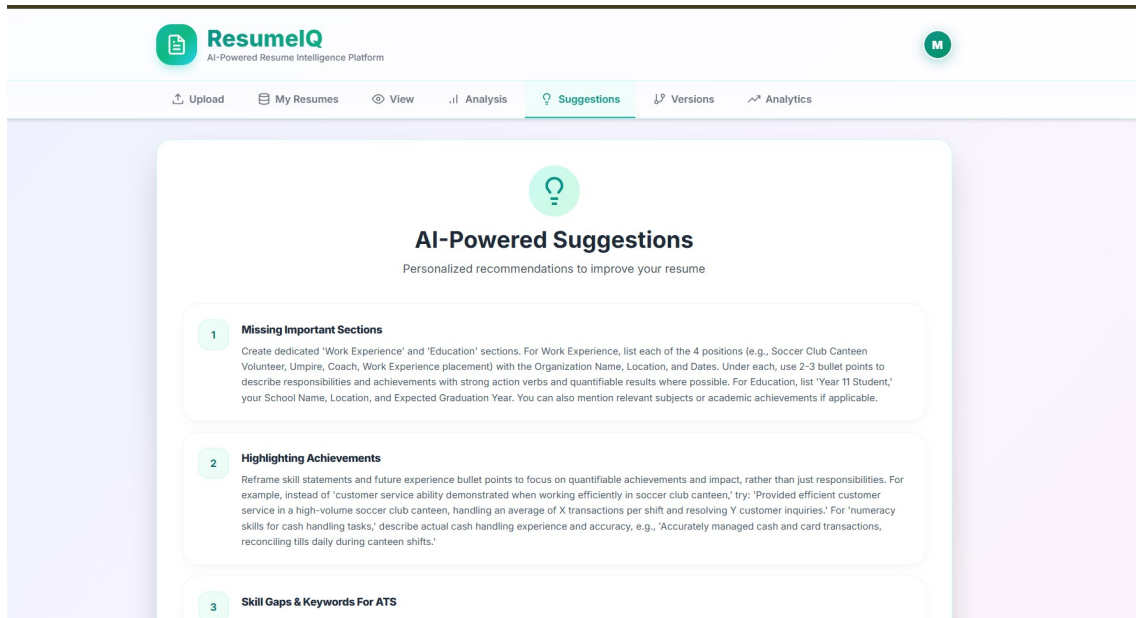


Fig. 6.8 Suggestions Page

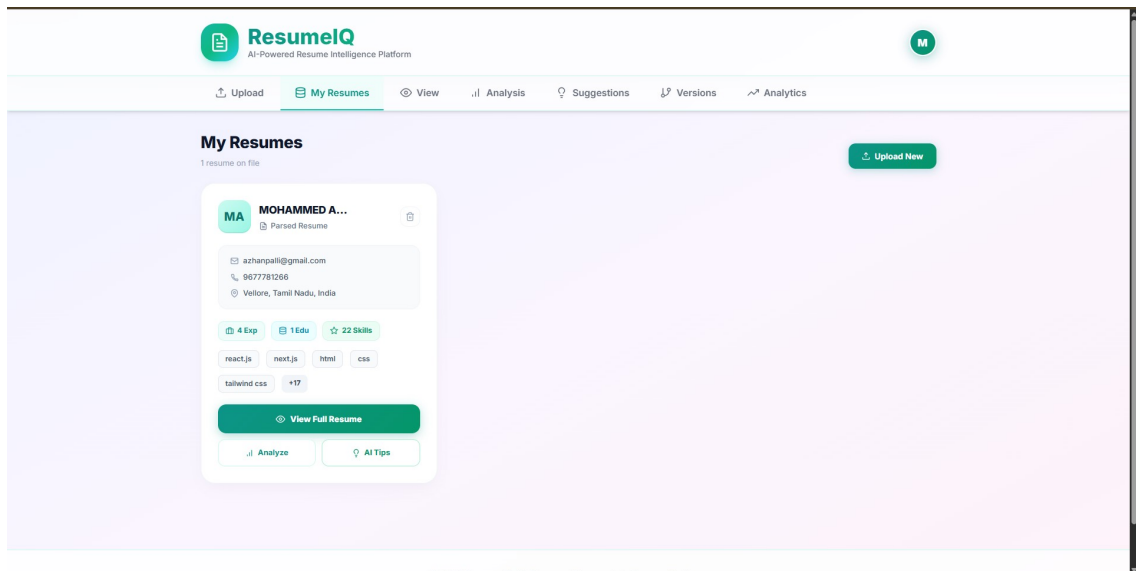


Fig. 6.9 Resumes History Page

System demonstration explains how the ResumeIQ system works in practice. In the beginning, the user loads the resume in the interface, as illustrated in Fig. 6.1. Resumes can be uploaded in two different file formats such as PDF and DOCX formats.

After loading, the system analyzes the resume, extracting structured information such as personal information, skills, educational qualifications, and work experience, as shown in Fig. 6.2 and Fig. 6.3. This explains the efficiency of the system in extracting structured data from the unstructured resume data.

The resume is further evaluated based on its score obtained from the ATS algorithm, and score along with the feedback message is displayed, as seen in Fig. 6.3. This helps the user in judging the resume quality.

Finally, the system suggests some recommendations for improving the resume by detecting missing skills and making the resume clear, as illustrated in Fig. 6.8.

6.4 Sample API / Code Snippet

This section presents sample API endpoints and code snippets used in the system to demonstrate implementation details.

6.4.1 Api calling

```
try {
  const interval = setInterval(() => {
    setUploadProgress(prev => Math.min(prev + 10, 90));
  }, 200);

  const response = await fetch(`${API_BASE_URL}/parse-resume/`, {
    method: 'POST',
    body: formData,
  });

  clearInterval(interval);
  setUploadProgress(100);
```

```

    if (response.ok) {
      const data = await response.json();
      setSelectedResume(data);
      setTimeout(() => {
        setActiveTab('view');
      }, 500);
    } else {
      alert('Error uploading resume');
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Error uploading resume');
  } finally {
    setLoading(false);
    setTimeout(() => setUploadProgress(0), 1000);
  }
}

```

6.4.2 REST Method

```

@app.post("/parse-resume/", response_model=schemas.ResumeData, tags=["Resume
Parsing"])
async def parse_and_save_resume(file: UploadFile = File(...), db: Session =
Depends(get_db)):
    """
    Upload a resume file (PDF or DOCX), parse it, save the result to the database,
    and return the structured content.
    """
    if not file.content_type in ["application/pdf", "application/vnd.openxmlformats-
officedocument.wordprocessingml.document"]:
        raise HTTPException(status_code=400, detail="Unsupported file type. Please upload a
PDF or DOCX file.")

```

```

file_bytes = await file.read()
raw_text = ""

if file.content_type == "application/pdf":
    raw_text = extract_text_from_pdf(file_bytes)
else:
    raw_text = extract_text_from_docx(file_bytes)

if not raw_text.strip():
    raise HTTPException(status_code=400, detail="Could not extract text from the
document.")

structured_data = await parse_resume_with_gemini(raw_text)
crud.create_or_update_resume(db=db, resume_data=structured_data)
return structured_data

```

6.5 Detailed Output Analysis

Table 6.1 System Output Summary

Parameter	Output
Resume Format	PDF / DOCX
Extracted Fields	Personal Info, Skills, Education, Experience
Processing Time	5–12 seconds (depending on server latency)
Score Range	0–100
Suggestions	Context-aware AI recommendations
Versioning Support	Multiple versions with dynamic URLs

Various outputs are produced by the ResumeIQ system when analyzing a resume. These outputs include data extraction, resume scoring, and suggestions by artificial intelligence.

Data extraction involves extracting essential details from the resume, which includes names, email addresses, contact numbers, skills, education, projects, and previous work experiences. This indicates that the system has a capability of converting unstructured data into structured data format.

Resume scoring involves analyzing a resume based on certain parameters, which include relevancy of skills, readability, and proper grammar usage. In this case, the output will be the overall score (Score). The overall score (Score) is computed using S_{skills} , S_{read} , and S_{grammar} .

Apart from the score, other outputs produced by the system include suggestions that can help improve the resume. These suggestions may include adding certain skills, enhancing clarity, and structuring the content. Finally, the system displays its output using a dashboard.

CHAPTER 7

RESULT AND DISCUSSION

Results and Discussion section provides the results that were achieved through the use of the ResumeIQ system. In this chapter, the performance of the developed system is examined in relation to its parsing ability, scoring efficiency, and generation of suggestions.

The performance of the system has been tested by implementing it on different resumes in order to understand its capabilities of parsing, scoring, and generating feedback. The findings that have been observed in this regard are presented below.

7.1 System Performance Evaluation

Table 7.1 ResumeIQ System Performance Metrics

Metric	Result	Description
Resume Parsing Accuracy	~90-98%	Correct extraction of structured data
Skills Extraction Accuracy	High Accuracy	Relevant skills identified correctly
Response Time	5–12 seconds (depending on server latency)	Fast API processing
ATS Score Consistency	Stable	Consistent scoring output
Suggestion Quality	Good	Meaningful AI recommendations

The ResumeIQ system was tested based on various metrics to assess its efficiency in resume processing. The ResumeIQ system showed excellent efficiency in identifying structured data like skills, education, and employment history.

The NLP component was effective at finding useful skills and context. The response time of the system ranges between 5–12 seconds depending on server latency which is satisfactory.

The scoring system based on the ATS proved to be quite consistent, and recommendations provided by AI were quite relevant and helpful. All the mentioned characteristics of the system show that ResumeIQ will successfully perform resume analysis tasks.

Table 7.2 Comparison with Existing Systems

Feature	Existing System	ResumeIQ
Resume Parsing	Manual	Automated
Accuracy	Medium	High
Skill Matching	Keyword-based	AI-based
Suggestions	Not available	Available
Processing Time	Slow	Fast

7.2 System Usability

Usability of ResumeIQ was assessed based on the interaction of users and the interface design of the system. The system offers a user-friendly interface that enables users to upload resumes and see the result of analysis effortlessly.

Users found navigating through various components of the system, including resume upload, extracted data, scores, and suggestions, easy and convenient. The way the results are presented and structured makes the dashboard more user-friendly.

The addition of AI-generated suggestions makes the system more usable as users can improve their resumes based on the suggestions provided. Overall, the system is easy to use for both recruiters and job seekers.

7.3 Cost Analysis

Table 7.3 Project Cost Analysis

Resource	Tool / Technology	Cost	Remarks
Frontend	React	Free	Open-source
Backend	FastAPI	Free	Open-source
Database	SQLAlchemy	Free	Local/Cloud
AI Processing	NLP Models	Free	Open-source
Development Tools	VS Code, GitHub	Free	Student tools
Total Cost	—	₹0	Fully open-source

ResumeIQ is an entirely open-source based solution, thus costing nothing directly. Tools like React, FastAPI, and SQLAlchemy are among those used for building the software, and they are free to use.

Using free NLP algorithms and models helped cut down the cost of the project even further. Similarly, development tools like Visual Studio Code and GitHub have been used at no cost. This economical way of building a solution shows how costly it is not to build a sophisticated system based on AI technology.

7.4 Discussion

The results produced by the ResumeIQ software illustrate the viability of utilizing both AI and NLP in the resume analysis process. Thus, the tool is capable of providing accurate insights into unstructured resumes.

From the perspective of evaluation metrics, it can be noted that the system is capable of accurately parsing data presented in resumes while correctly identifying relevant skills. The automatic ATS-based scoring helps users get a clear understanding of their strengths and weaknesses. In terms of usability, the system proves to be an intuitive solution with easy navigation. At the same time, it integrates useful AI-driven recommendations to help users improve their resumes. Concerning cost factors, it should be noted that the entire solution relies on free technology. Still, some limitations are observed, including low levels of accuracy for particularly unstructured resumes and a dependence on certain skill sets.

Therefore, on a general basis, the ResumeIQ system helps make resume analysis more efficient and accurate. User authentication is an important aspect in ensuring that access to the resume information is secure and personal to the user. The use of caching technology has greatly improved processing times because no resume is analyzed more than once. Also, resume versioning means that the user will be able to develop several custom versions of the resume and then send it via different public links. The system shows great potential for application in the real world.

CHAPTER 8

SUMMARY

8.1 Summary of Findings

ResumeIQ illustrates how AI and NLP can be effectively leveraged to automatically process resumes and assess candidates. The system is able to extract information from unstructured resumes, creating a structured database consisting of personal information, skills, education, projects, and employment history.

A score based on skill-relatedness, readability, and grammatical correctness is assigned using the ATS. In addition, AI recommendations enable users to optimize their resumes, filling any gaps in terms of skills and quality.

User authentication guarantees security and personalized access to the resume database. Multiple customized resumes can be generated via resume versioning, sharing them via unique URLs. File hashing and caching are employed as optimization techniques, which minimize processing time by preventing re-analysis of identical resumes.

In conclusion, ResumeIQ is a highly scalable and efficient system, automating recruitment procedures. The current project holds great promise for practical implementation.

8.2 Recommendations

From the findings and the overall performance of the ResumeIQ system, there are some recommendations that can be made to improve the performance of the system in the future. First of all, the system should allow the use of resumes written in different languages. This would improve the performance of the ResumeIQ system since it will be able to accommodate more customers.

The second recommendation is improving the precision of the algorithms used in the artificial intelligence models. The algorithms should enable the system to gain more knowledge about the resume being evaluated.

The third recommendation is the addition of sophisticated analytics and visualization features in the ResumeIQ system. This will enable users to get more insights into the

performance of their resumes. Finally, the cloud-based deployment and scalability should be improved in the system.

8.3 Conclusion

In the case of the ResumeIQ project, one can clearly see how AI and NLP technologies can be applied to real-life applications. Thus, this tool allows users to easily analyze resumes, scoring them and giving appropriate suggestions, thus saving recruiters' time.

The combination of ATS-based resume analysis and AI-powered recommendation gives recruiters additional value and valuable insight into candidates' resumes. At the same time, the fact that the resume data can be converted into structured data shows how efficient this solution is.

Also, the use of modern technologies like React and FastAPI made it possible to create a truly scalable and efficient system. Another important point is that even AI-based projects can be created with the help of open-source tools. All in all, ResumeIQ proved to be a robust and intelligent solution for resume analysis. This solution has the potential of being applied to recruitment systems.

CHAPTER 9

REFERENCES

- [1] Abisha D., Keerthana S., Navedha Evanjaline R., Kavitha K., Jothi Mary S., Ramya R., “Resspar: AI-Driven Resume Parsing and Recruitment System using NLP and Generative AI,” in Proc. 2024 Second Int. Conf. Intelligent Cyber Physical Systems and Internet of Things (ICoICI), IEEE, 2024.
- [2] Bharadwaj S., Rudra Varun, Potukuchi Sreeram Aditya, Macherla Nikhil, and G. Charles Babu, “Resume Screening using NLP and LSTM,” in Proc. 2022 Int. Conf. Inventive Computation Technologies (ICICT), IEEE, July 2022.
- [3] Pabalkar S., Patel P., Choudhary R., Panoch V., Yadav S., and Ghogale H., “Resume Analyzer Using Natural Language Processing (NLP),” in Proc. 2024 Int. Conf. Intelligent Systems and Advanced Applications (ICISAA), IEEE, 2024.
- [4] Novaković N. and Dražeta L., “Applicant Tracking System: A Powerful Recruiters’ Tool,” in Proc. Sinteza 2024 - Int. Scientific Conf. Information Technology, Computer Science, and Data Science, 2024.
- [5] Varun Sai Ganga, S. Kranthi, Sri Navya Deepika Regalla, and Lakshmi Lokeshwar Vissakodeti, “Skill Extractor in Resumes using Machine Learning and Streamlit,” in Proc. 2024 Int. Conf. Trends in Engineering Systems and Technologies (ICTEST), IEEE, 2024.
- [6] Ali I., Mughal N., Khand Z. H., Ahmed J., and Mujtaba G., “Resume Classification System using Natural Language Processing and Machine Learning Techniques,” Mehran Univ. Res. J. Eng. Technol., vol. 41, no. 1, 2022.
- [7] Mohit B., “Named Entity Recognition,” in Natural Language Processing of Semitic Languages, Springer, 2014.
- [8] Singh S., Jadhav I., Waghmare V., and Ramesh R., “Artificial Intelligence Recruitment System,” Int. J. Eng. Appl. Sci. Technol., vol. 5, no. 3, 2020.

- [9] Pudasaini S., Shakya S., Lamichhane S., Adhikari S., Tamang A., and Adhikari S., “Application of NLP for Information Extraction from Unstructured Documents,” Lecture Notes in Networks and Systems, Springer, 2022.
- [10] Zaman G., Mahdin H., Hussain K., and Rahman A., “Information Extraction from Semi and Unstructured Data Sources: A Systematic Literature Review,” ICIC Express Lett., vol. 14, no. 6, 2020.